

Nodejs session 2

▼ Coding

```
#!/usr/bin/env node

import { Command } from "commander";
import inquirer from 'inquirer';
import Fs from 'fs';

const Program = new Command();

const filePath = './courses.json';

const questions = [
  {
    type: 'input',
    name: 'title',
    message: 'What is the title of the course?',
  },
  {
    type: 'input',
    name: 'Price',
    message: 'What is the price of the course?',
  }
]

Program
  .name('codeZone-courses-manager')
  .version('1.0.0')
  .description('CLI to make courses');

Program.command('add')
  .alias('a')
  .description('Add a new course')
```

```

    .action((param, options) => {
        inquirer
    })
    .prompt(questions)
    .then((answers) => {
        console.log(answers);
        if (Fs.existsSync(filePath)) {
            Fs.readFile(filePath, 'utf8', (err, fileContent) =>
            {
                if (err) {
                    console.error('Error reading file:', err);
                    process.exit();
                }

                console.log("File content --->", fileContent);

                let fileContentAsJson;

                try {
                    fileContentAsJson = JSON.parse(fileContent);

                    // 📌 لو منش Array
                    if (!Array.isArray(fileContentAsJson)) {
                        fileContentAsJson = [fileContentAsJson];
                    }

                } catch (e) {
                    // 📌 بايظ أو فاضي JSON لو
                    fileContentAsJson = [];
                }

                fileContentAsJson.push(answers);

                Fs.writeFile(
                    filePath,
                    JSON.stringify(fileContentAsJson, null, 2),
                    'utf8',

```

```

        () => {
            console.log('Course added successfully!');
        }
    );
});
} else {
    Fs.writeFile(filePath, JSON.stringify([answers]),
'utf8', () => {
        console.log('Course added successfully!');
    });
}
})
.catch((error) => {
    if (error.isTtyError) {
        // Prompt couldn't be rendered in the current environment
    } else {
        // Something else went wrong
    }
});
});
Program.command('list')
.alias('l')
.description('List all courses')
.action(() => {
    Fs.readFile(filePath, 'utf8', (err, Content) => {
        if (err) {
            console.error('Error reading file:', err);
            process.exit();
        }
        console.table(JSON.parse(Content));
    });
});

Program.parse(process.argv);

```

▼ This Is Explanation

🧠 1. File System (fs module) يعني إيه؟ 💡

مكتبة في Node.js بتخليك تتعامل مع الملفات:

- تقرأ
- تكتب
- تمسح
- تعدل

🔧 عملي:

```
import fs from 'fs';

// قراءة ملف
fs.readFile('data.json', 'utf8', (err, data) => {
  console.log(data);
});

// كتابة ملف
fs.writeFile('data.json', 'Hello', () => {
  console.log('done');
});
```

⚠️ مهم:

- لازم تحدد **encoding** (زي utf8)
- وإلا هيرجعلك **Buffer** مش نص

⚡ 2. Blocking vs Non-Blocking: الفكرة 🧠

بيحب السرعة وعدم التعطيل → **event loop** شغال بـ Node.js

❌ Blocking (بيوقف الدنيا)

```
const data=fs.readFileSync('file.txt');  
console.log(data);
```

👉 البرنامج كله يقف لحد ما الملف يخلص

✅ Non-Blocking (الصح)

```
fs.readFile('file.txt', (err,data) => {  
  console.log(data);  
});
```

👉 يكمل شغل عادي ويقرأ في الخلفية

الخلاصة: 💣

- Sync = خطر في production
- Async = الصح دائماً

abc 3. Encoding (مهمة جدًا) 💡

الملف بيتخزن bytes مش نص

مثال: 🛠️

```
fs.readFile('file.txt', (err,data) => {  
  console.log(data);// Buffer  
});
```

الحل:

```
fs.readFile('file.txt','utf8', (err,data) => {  
  console.log(data);// String  
});
```

خلاصة:

- utf8 = نص مفهوم

• بدونها = Buffer

4. Buffers يعني إيه؟

طريقة Node بيمثل بيها البيانات الخام (bytes)

مثال:

```
constbuffer=Buffer.from('Hello');  
console.log(buffer);// <Buffer 48 65 6c 6c 6f>
```

تستخدم امتى؟

• ملفات

- network
- streams

5. Streams: الفكرة:

بدل ما تقرأ الملف كله مرة واحدة
👉 اقرأه chunk chunk

مثال: 🛠️

```
const stream = fs.createReadStream('big.txt', 'utf8');

stream.on('data', (chunk) => {
  console.log(chunk);
});
```

ليه مهمة؟ 🔥

• ملفات كبيرة

- أعلى performance
- أقل memory

📦 6. NPM يعني إيه؟ 💡

libraries بتثبت بيه → Package Manager

أوامر مهمة: 🛠️

```
npm init -y
npm install lodash
npm install nodemon --save-dev
```

بيعمل إيه؟ 📁

• ينشئ `package.json`

⚠️ مهم:

- `dependencies` → في production
- `devDependencies` → في التطوير بس

🔧 7. Create CLI Tool؟ يعني إيه 💡

برنامج command line زي:

```
my-tool add
```

1. package.json 🔧 الخطوات:

```
{
  "name": "my-cli",
  "bin": {
    "my-cli": "./index.js"
  }
}
```

2. في index.js

```
#!/usr/bin/env node

console.log("Hello CLI");
```

3. اربطه بالجهاز

```
npm link
```

4. شغله

```
my-cli
```

نشره على npm 🚀

```
npm login  
npm publish
```

ربط كل الكلام ببعض (أهم جزء) 🧠

انت دلوقتي بتبني CLI:

- 📁 fs → بتخزن الكورسات
- ⚡ async → عشان الأداء
- 📄 encoding → عشان البيانات تبقى مفهومة
- 📦 npm → تثبيت commander / inquirer
- 🌊 streams → لو البيانات كبرت
- 🛠 CLI → المستخدم يتعامل مع البرنامج

الخلاصة السريعة 🔥

المفهوم	فايده
fs	تتعامل مع الملفات

المفهوم	فايده
async	سرعة وأداء
encoding	لنص bytes تحويل
buffer	تمثيل البيانات
streams	ملفات كبيرة
npm	إدارة المكتبات
CLI	زي المحترفين tools تعمل

لو عايز ليفل أعلى بقى: 🐱

أقدر أحول مشروعك ده لـ:

- كثير commands احترافي فيه CLI
- - validation + logging
- - security issues (bug bounty تتعلم منها)

قولّي:

"عايز أحوله لأداة احترافية"

ونفشحه سوا 🔥